

FOUNDRY [0/1] 0%

1.1 — Build a single self-contained HTML file at visualizer.htm...

PLANNER (opus) | 0m 37s | 27 events

PLANNER Output

```
[tool] Read — /Users/name/homelab/context-foundry/src/agent.rs
[tool] Read — /Users/name/homelab/context-foundry/src/main.rs
[tool] Read — /Users/name/homelab/context-foundry/src/config.rs
[tool] Read — /Users/name/homelab/context-foundry/src/task.rs
[tool] Read — /Users/name/homelab/context-foundry/src/studio.rs
[tool] Glob — src/*.rs
```

Now let me read the remaining source files.

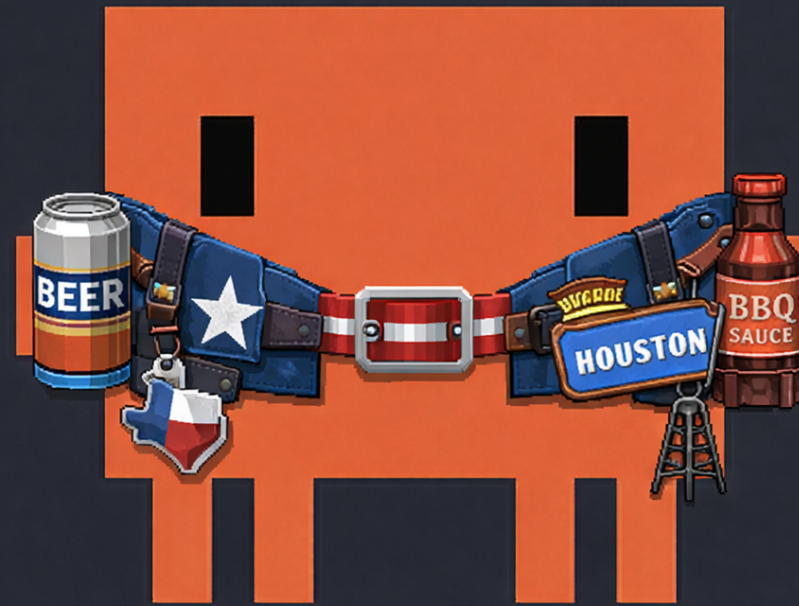
```
[tool] Read — /Users/name/homelab/context-foundry/src/studio.rs
[tool] Glob — src/*.rs
[tool] Read — /Users/name/homelab/context-foundry/src/studio.rs
[tool] Read — /Users/name/homelab/context-foundry/src/git.rs
[tool] Read — /Users/name/homelab/context-foundry/src/patterns.rs
[tool] Read — /Users/name/homelab/context-foundry/src/tui.rs
[tool] Read — /Users/name/homelab/context-foundry/src/update.rs
[tool] Read — /Users/name/homelab/context-foundry/src/utils.rs
[tool] Read — /Users/name/homelab/context-foundry/src/prompts.rs
```

Now let me read the rest of studio.rs to understand the full studio flow.

```
[tool] Read — /Users/name/homelab/context-foundry/src/studio.rs
[tool] Grep — fn (scan_project|run_tui|prepare_workspace|launch_session|capture_artifacts|smooth_
[tool] Grep — fn (launch_sessions|compose_smoothed_prompt|capture_artifacts|build_execution_brief
[tool] Read — /Users/name/homelab/context-foundry/src/studio.rs
```

Log

```
04:29:19 Loop started — 1 tasks (0 done, 1 pending)
04:29:19 Task 1.1 started
04:29:19 Matched 45 patterns for task
04:29:19 PLANNER spawned (opus)
```



AI TINKERERS

MAIN STAGE SPEAKER



CONTEXT FOUNDRY: RUST AI ORCHESTRATOR

Silviu

Houston • March 10, 2026

/Users/name/homelab/context-foundry/src/config.rs sk ↑↓ scroll

Context foundry

Orchestrating AI Agents in RUST

Build loops, execution contracts and multi-model comparison

Silviu Nedea, 10-March, 2026

The problem

- The goal is autonomous coding time, even while I sleep.
- Model output is probabilistic, not deterministic.
- In a long-running loop, low-quality steps can compound if nothing catches them.
- So the system needs validation, recovery, and memory.
- When it reaches the planned objective, it should discover adjacent work in the same context.

Credit



Geoff Huntley

Ralph loop



Dex Horthy

Context Engineering



Claude Code

About me



Silviu Nedea

AI Tinkerer

- Building since Claude 3.5 Sonnet in July 2024
- Before Claude Code, coding was rough, but we did it.
- My first AI experience: minecraft bots
- My first app: AI Flashcard Generator
- Open AI o1 September 12, 2024
- Figured o1 could only build up to ~700 loc reliably
- Over 100 apps and agents
- Context Foundry, October 2, 2025, Python + MCP Server
- 15 months ago

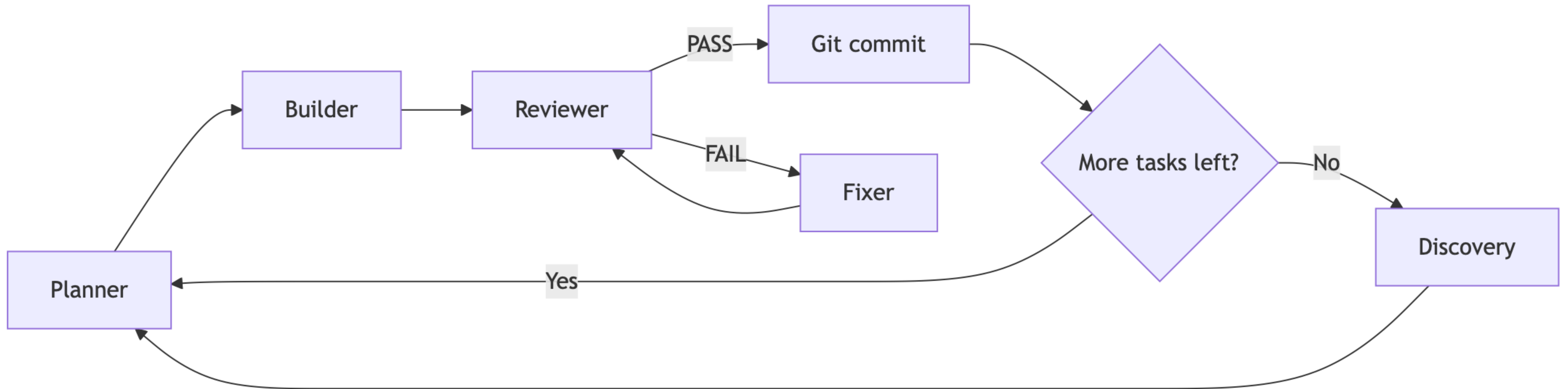
Context-Foundry

Two modes

	Build Loop	Studio
Mode	Fully autonomous	Interactive
Input	Markdown checklist	Freeform Prompt
Agents	Planner/Builder/Reviewer/Fixer	Claude Code, Codex CLI, etc.
Output	Committed code	Side-by-side comparison

Part 1: the build loop

My own Ralph



DEMO

Task format: impl_plan.md

Role specific prompts

- Planner — writes `.buildloop/current-plan.md`
 - Dependencies
 - File operations
 - Full type signatures, verification steps
 - “Write for machine consumption. Eliminate ALL ambiguity.”
- Builder — executes plan literally:
 - “Follow the plan precisely. Do not deviate, interpret, or add”

Why machine-friendly plans

- Machine-friendly plans are deterministic:
 - extract file paths, operation types (CREATE / MODIFY)
 - full function signatures and type definitions up front
- The plan becomes a machine-readable contract, not a suggestion.
- Fewer judgement calls = fewer compounding errors across tasks

The review gate

- Reviewer writes structured JSON filings (high / medium / low)
- Convergence: PASS AND high = 0 AND medium = 0
- Up to 2 review passes; one fixer attempt between them
- Commits: feat (T1.1) on pass, WIP (T1.1) on fail.
- WIP means: keep progress, but do not trust it yet.

Pattern learning

- Patterns are how the system gets better over time.
- Each pattern has: keywords, frequency, issue, solution, auto_apply
- Scoring: word match +2 substring +1, tech stack +1, auto_apply +2
- Top 10 matched patterns injected into planner/reviewer prompts
- Global by default. Frequency 3+: always inject.

Part 2: studio

Interactive multi-model Ralph

STUDIO /Users/name/homelab/secondbrain
providers=both workspace=isolated contract=Custom Contract 03:10:29 editor=nano claude=opus (ready) codex=<cli-default> (ready)
keys: e edit c cycle contract v cycle editor a add d delete s start f follow-up tab/shift-tab focus click pane drag split bars resize

Project Scan

scan: 2026-03-07 03:33:49 UTC
stack: Docker Compose
top: .DS_Store, .buildloop, .claude, .env, .env.example, .fo
data: .buildloop/audit-report.md, .buildloop/logs/AUDITOR-20

Prompt (editing)

determine the top 5 improvements to make to secondbrain.
dont implement them, write out a report

Execution Contracts

selected: Custom Contract 03:10:29
> Custom Contract 03:10:29
Standard Build Contract
vars: {{workspace_dir}} {{artifact_dir}} {{provider_label}}
editor: nano (press v to change)

Execution Brief

You are running inside Foundry Studio through the Claude + Codex CLI.

User objective:

determine the top 5 improvements to make to secondbrain.
dont implement them, write out a report

Sessions

| Claude running 8ev 20s
>| Codex running 10ev 20s

Output [Codex]

```
sort"  
[:item.started] /bin/zsh -lc "pwd && rg --files -g 'package.json' -g  
'tsconfig.json' -g 'Dockerfile*' -g 'docker-compose*' -g 'README*' -g  
'ARCHITECTURE.md' -g 'IMPL_PLAN.md' -g '.buildloop/**' -g 'frontend/**' -g  
'backend/**' -g 'worker/**' -g 'api/**' -g 'server/**'"
```

```
/bin/zsh -lc 'git status --short'
```

```
/bin/zsh -lc "pwd && rg --files -g 'package.json' -g 'tsconfig.json' -g  
'Dockerfile*' -g 'docker-compose*' -g 'README*' -g 'ARCHITECTURE.md' -g  
'IMPL_PLAN.md' -g '.buildloop/**' -g 'frontend/**' -g 'backend/**' -g  
'worker/**' -g 'api/**' -g 'server/**'"
```

Artifacts + Log

Claude: ready - authenticated; cached live smoke OK (330s old)
Codex: ready - authenticated; cached live smoke OK (328s old)
contract: Custom Contract 03:10:29
workspace: /Users/name/homelab/secondbrain/.foundry/studio/workspaces/codex
brief: ...ex/.foundry/studio/artifacts/20260307-033454/codex/execution-brief.md
artifacts: ...udio/workspaces/codex/.foundry/studio/artifacts/20260307-033454/c
started: 03:34:54 UTC

Execution contracts

- Markdown templates stored in `.foundry/studio/contracts/`:
- `# Standard Build Contract`
 - You are `{{provider_label}}` working in `{{workspace_dir}}`
- `## Rules`
 - Write artifacts to `{{artifact_dir}}`
 - Run tests before declaring success
 - Do not modify files outside the workspace
 - Each contract can have an `.attachments.json` sidecar

Attachments

- Two modes:
 - 1. Inline File
 - 2. Directory Tree
- Injected between contract and project scan:
 - — BEGIN ATTACHMENT: docs/api.md —
 - <file content>
 - — END ATTACHMENT: docs/api.md —

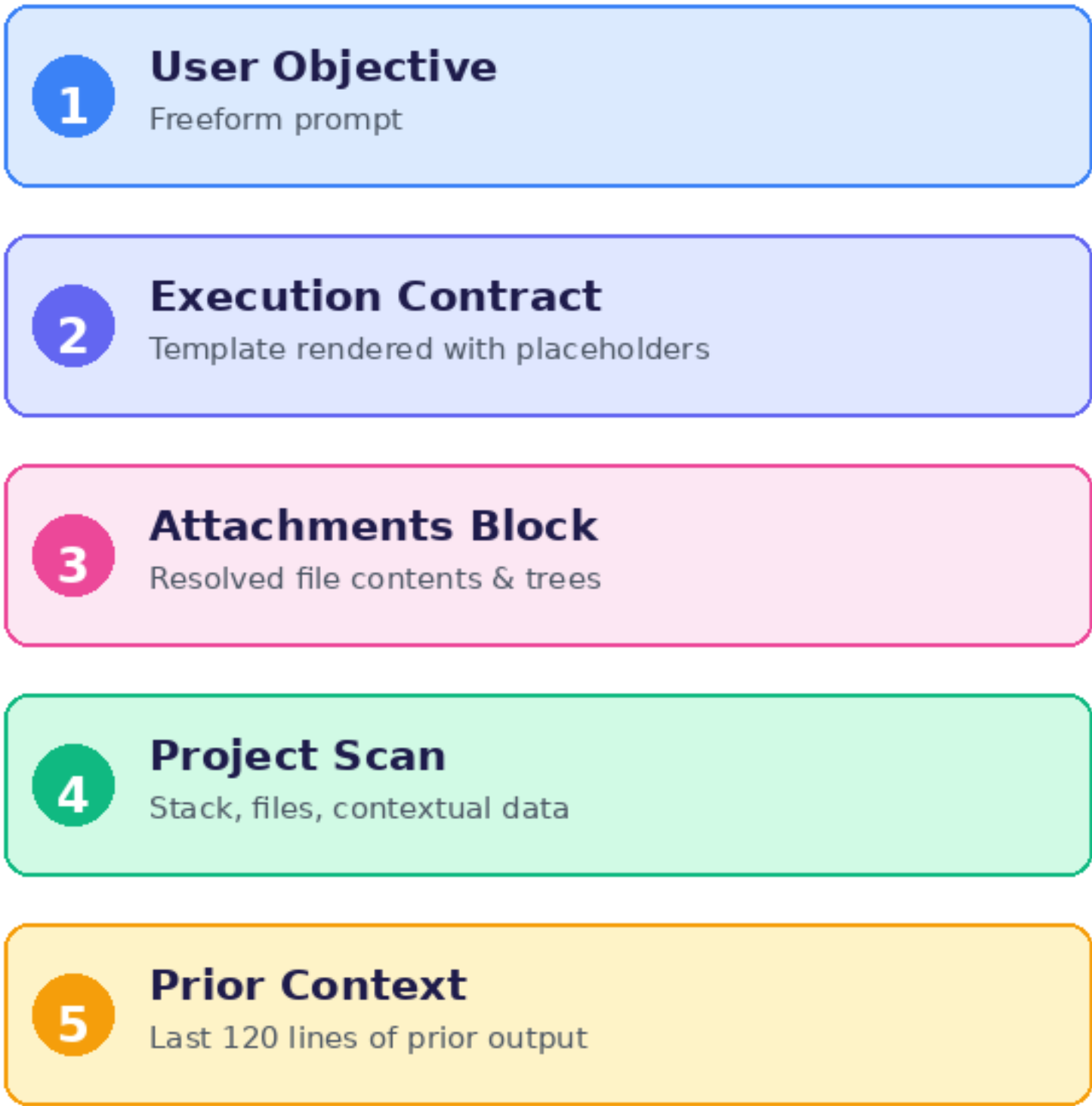
Prompt assembly

- `compose_prompt()`
- 1. User objective (freeform prompt)
- 2. Execution contract (template rendered)
- 3. Attachments block (resolved files/trees)
- 4. Project scan (stack, files, data)
- 5. Prior context (follow-ups: last 120 lines)
- Saved to disk as `execution-brief.md` before agent starts.
- Preview rendered line in the Execution Brief
- Follow-ups reuse the same workspace and get tail of prior output, so that agent can reference it's own previous decisions.

compose_prompt()

Execution Brief Assembly

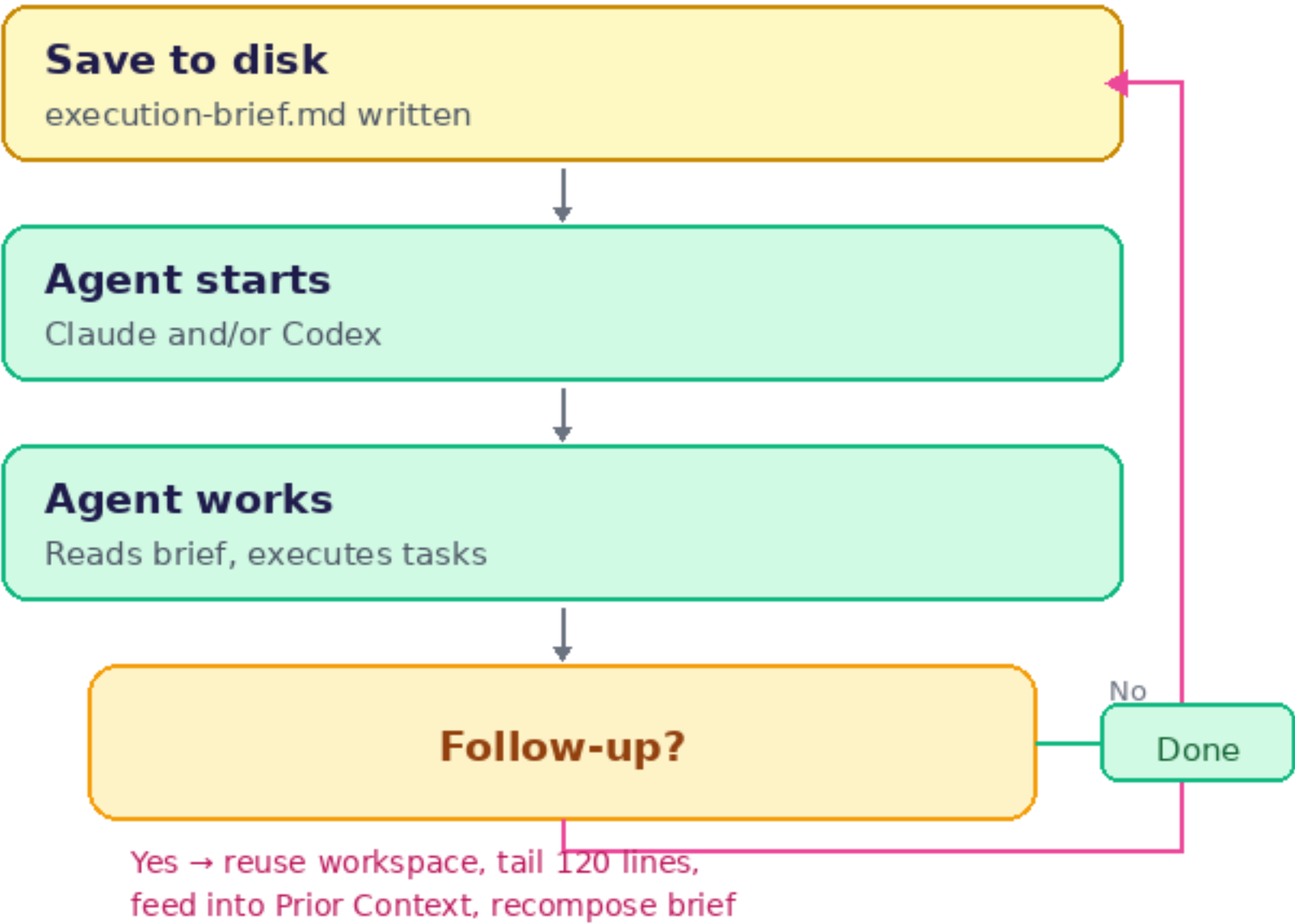
PROMPT SECTIONS



EXECUTION BRIEF



AGENT PIPELINE



Multi-model comparison

- In isolated mode with Both:
 - Same prompt, separate workspace copy per agent
 - Both run as separate tasks simultaneously
 - Output streams to same TUI
 - Compare the results, pick the best output

Architecture decisions

- Why Rust?
 - PTY management, async spawning and TUI rendering need low-level control
 - ratatui for the terminal UI
 - tokio for async agent tasks + event channels
- Why isolated workspaces?
 - Agents can clobber files without affecting each other
 - True A/B comparison of model outputs
 - Follow-ups preserve state from prior runs

What I'd do differently

- Workspace snapshots are expensive, when you copy the whole project tree. Git work trees would be lighter.
- Pattern matching is keyword based, switching to embeddings would match better but adds complexity
- Codex transport stalls - websocket reconnection is fragile. Retry logic helps but isn't a real fix.

Links

- Context Foundry: <https://github.com/context-foundry/context-foundry>
- Ralph concept: <https://ghuntley.com/ralph/>
- HumanLayer/CodeLayer: <https://github.com/humanlayer/humanlayer>
- Mindflayer: <https://github.com/PrismarineJS/mineflayer>